

Count Divisors

$n=24$ (1, 2, 3, 4, 6, 8, 12, 24)

OP = 8

$n=100$ (1, 2, 4, 5, 10, 20, 25, 50, 100)

OP = 9

```
static int countDivisors(int n){
    //code here
    int cnt=0;
    for(int i=1; i<=n; i++){
        if(n%i==0) cnt++;
    }
    return cnt;
}
```

→ n iteration

10^8 iteration = 1 sec

1 iteration = $\frac{1}{10^8}$ sec

$n=10^3 \Rightarrow 10^3 \times \frac{1}{10^8} = \frac{1}{10^5}$ secs

$n=10^8 \Rightarrow 1$ sec

$n=10^9 \Rightarrow 10^9 \times \frac{1}{10^8} = 10$ secs

$n=10^{18} \Rightarrow 10^{18} \times \frac{1}{10^8} = 10^{10}$ secs

317 years.

$n=24$ — 1, 2, 3, 4, 6, 8, 12 — 24 — cnt=1

$n=64$ — 1, 2, 4, 8, 16, 32 — 64 — cnt=1

$n=100$ — 1, 2, 4, 5, 10, 20, 25, 50 — 100 — cnt=1

```
static int countDivisors(int n){
    //code here
    int cnt=1;
    for(int i=1; i<=n/2; i++){
        if(n%i==0) cnt++;
    }
    return cnt;
}
```

<u>i</u>	<u>n=24</u>	<u>n/i</u>	<u>cnt=0</u>
1	<=	24	+ 2
2	<=	12	+ 2
3	<=	8	+ 2
4	<=	6	+ 2
6		4	
8		3	
12		2	
24		1	

<u>i</u>	<u>n=100</u>	<u>n/i</u>	<u>cnt=0</u>
1	<=	100	+ 2
2	<=	50	+ 2
4	<=	25	+ 2
5	<=	20	+ 2
10	<=	10	+ 1
20		5	
25		4	
50		2	
100		1	

<u>i</u>	<u>n=28</u>	<u>n/i</u>	<u>cnt=0</u>
1	<=	28	+ 2
2	<=	14	+ 2
4	<=	7	+ 2
7		4	
14		2	
28		1	

$i \leq n/i$


```
static int countDivisors(int n){
    //code here
    int cnt=0;
    for(int i=1;i<=n/i;i++){
        if(n%i==0){
            if(i==n/i) cnt++;
            else cnt+=2;
        }
    }
    return cnt;
}
```

$i \leq n/i$
 $i^2 \leq n$
 $i \leq \sqrt{n}$

$\sqrt{n}$
 $n = 10^{16} \Rightarrow \sqrt{10^{16}} \Rightarrow 10^8$
 $10^8 \approx 1 \text{ sec}$

Power of optimization

Prime Number

<u>i</u>	<u>n=24</u>	<u>n/i</u>	<u>cnt=0</u>
1	<=	24	+ 2 -
2	<=	12	+ 2 -
3	<=	8	+ 2
4	<=	6	+ 2

```
class Solution {
    static boolean isPrime(int n) {
        // code here
        int cnt=0;
        for(int i=1;i<=n/i;i++){
            if(n%i==0){
                if(i==n/i) cnt++;
                else cnt+=2;
            }
        }
        return cnt==2;
    }
}
```

Perfect Number L.c 507

<u>i</u>	<u>n=28</u>	<u>n/i</u>	<u>Sum=0</u>
✓ 1	<=	✓ 28	29
✓ 2	<=	✓ 14	45
✓ 4	<=	✓ 7	56 - 28 = 28

```
bool checkPerfectNumber(int n) {
    int sum=0;
    for(int i=1;i<=n/i;i++){
        if(n%i==0){
            if(i==n/i) sum=sum+i;
            else sum=sum+i+(n/i);
        }
    }
    sum=sum-n;
    return sum==n;
}
```

Print Digits in reverse

$n = 1234$
 OP: 4
 3
 2
 1

<u>n</u>	<u>Ld</u>
1234 % 10 = 4	4
123 % 10 = 3	3
12 % 10 = 2	2
1 % 10 = 1	1
0	

$1234 \% 10 = 4$
 $1234 \% 100 = 34$
 $286 / 10 = 28$
 $286 / 100 = 2$


```
int n=scn.nextInt();
while(n!=0){
    int LD=n%10;
    System.out.println(LD);
    n=n/10;
}
System.out.println(n);
```

with input destruction.

✗ → you are not allowed to copy the given input.

n

1234 $n \% 10 = 4 / 1 = 4$

1234 $n \% 100 = 34 / 10 = 3$

1234 $n \% 1000 = 234 / 100 = 2$

1234 $n \% 10000 = 1234 / 1000 = 1$

div=1, 10, 100, 1000

$(n \% (div * 10)) / div$

div n

1 ≤ 100 $(n \% 10) / 1 = 0$

10 ≤ 100 $(n \% 100) / 10 = 0$

100 ≤ 100

```
int n=scn.nextInt();
int div=1;
while(div<=n){
    int LD=(n%(div*10))/div;
    System.out.println(LD);
    div=div*10;
}
System.out.println(n);
```

with out Input destruction.

Sum of Digits

n = 12345

OP: 15 (5+4+3+2+1)

```
int sum=0, div=1;
while(div<=n){
    int LD=(n%(div*10))/div;
    sum=sum+LD;
    div=div*10;
}
return sum;
```

n = 1234

OP: 4

count Digits

div n cnt=0

1 ≤ 1234 1

10 ≤ 1234 2

100 ≤ 1234 3

1000 ≤ 1234 4

```
int cnt=0, div=1;
while(div<=n){
    cnt++;
    div=div*10;
}
return cnt;
```


Armstrong Number

$$n = \underline{153} = 3$$

$$\text{OP: } 3^3 + 5^3 + 1^3 \\ 27 + 125 + 1 = \underline{153}$$

$$n = \underline{1634} = 4$$

$$\text{OP: } 4^4 + 3^4 + 6^4 + 1^4 \\ = 1634$$

user defined function

```
static boolean armstrongNumber(int n) {  
    // code here  
    int nod=countDigits(n);  
    int sum=0,div=1;  
    while(div<=n){  
        int LD=(n%(div*10))/div;  
        sum=sum+(int)Math.pow(LD,nod);  
        div=div*10;  
    }  
    return n==sum;  
}
```

$$n = 1234$$

$$\text{OP} = 4321$$

Reverse the Number

$$\begin{array}{l} \underline{4} \\ 1234 \quad 4 \cdot 10 + 4 = 4 \\ 1234 \quad 3(4 \cdot 10 + 3) \quad 43 \\ 1234 \quad 2(43 \cdot 10 + 2) \quad 432 \\ 1234 \quad 1(\underline{432} \cdot 10 + \underline{1}) \quad 4321 \end{array}$$

$$\boxed{\text{rev} = \text{rev} * 10 + \text{LD}}$$

Number Palindrome

```
int rev=0,div=1;  
while(div<=n){  
    int LD=(n%(div*10))/div;  
    rev=rev*10+LD;  
    div=div*10;  
}  
return rev==n;
```

Factorial =

$$n = 5$$

$$\boxed{5! = 4! \times 5}$$

$$\text{OP} = 120$$

$$(1 \times 2 \times 3 \times 4 \times 5)$$

$$i=1$$

$$\text{fact} = \text{fact} * i = 1$$

$$\underline{i=2}$$

$$\text{fact} = 1 \times 2 = 2$$

```
int n=scn.nextInt();  
int fact=1;  
for(int i=1;i<=n;i++) fact=fact*i;  
System.out.println(fact);
```

$$\underline{i=3}$$

$$\text{fact} = 2 \times 3 = 6$$

$$i=4$$

$$\text{fact} = 6 \times 4 = 24$$

$$i=5$$

$$\text{fact} = 24 \times 5 = 120$$

Pow

$$x = 2$$

$$n = 5$$

$$\text{OP} = 32$$

$$2^5 = 2 \times 2 \times 2 \times 2 \times 2$$

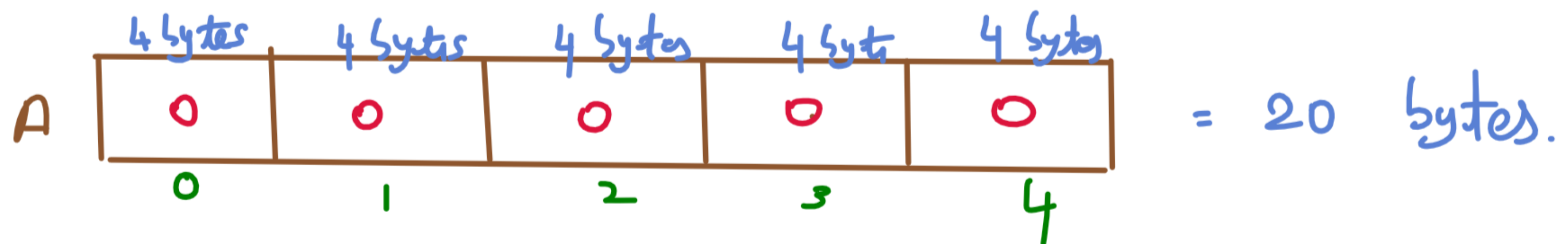
```
int x=scn.nextInt();  
int n=scn.nextInt();  
int pow=1;  
for(int i=1;i<=n;i++) pow=pow*x;  
System.out.println(pow);
```


Array — non Primitive

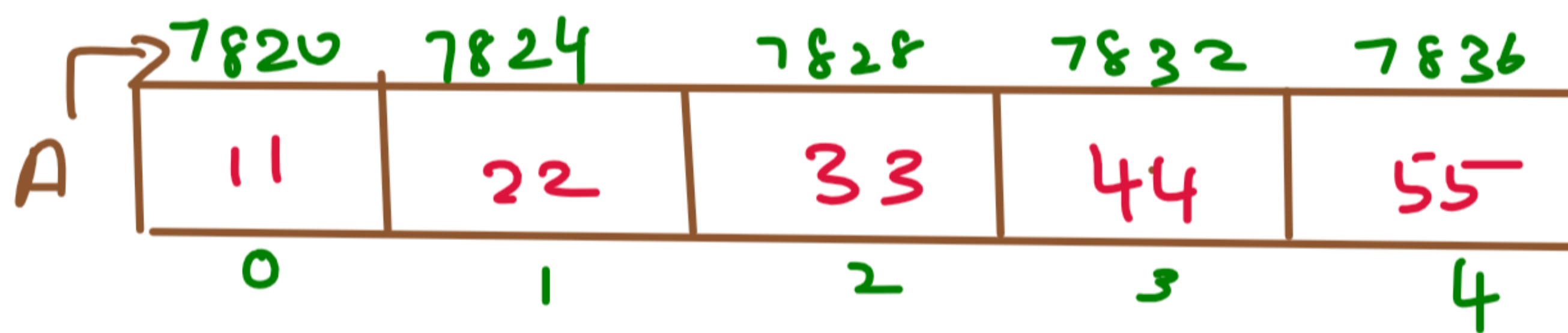
- used to store the Group of values with same data type.
- Syntax for declaring array.

data_type arr_name[] = new data_type[size];

int A[] = new int[5];



- Array Index always start from 0 to (n-1)



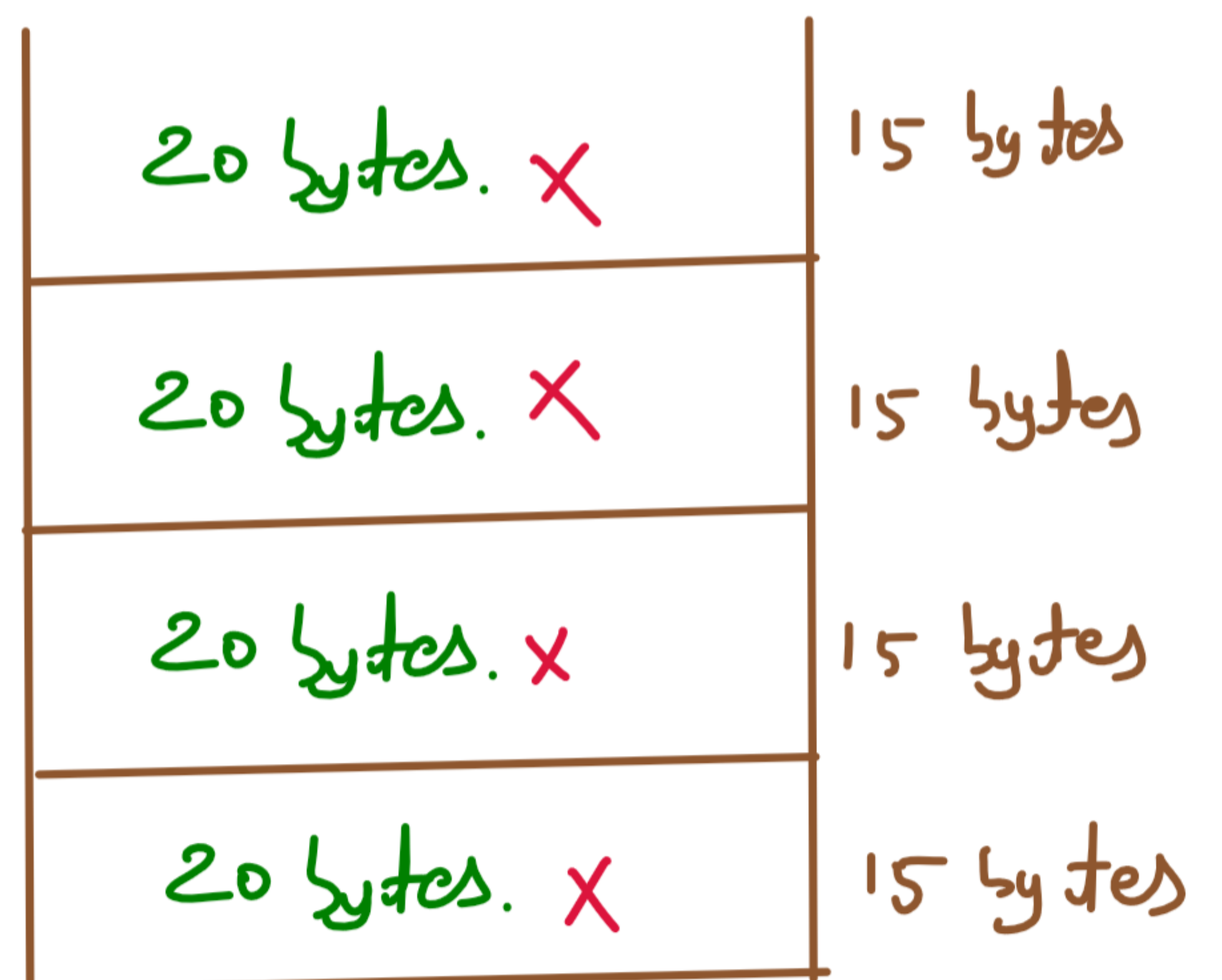
```
int A[] = new int[5];
A[0] = 11;
A[1] = 22;
A[2] = 33;
A[3] = 44;
A[4] = 55;
int B[] = {11, 22, 33, 44, 55};
System.out.println(A);
System.out.println(B);
```

- Array is the fixed size

- Array is the continuous memory allocation.

Run Time Initialization

```
public static void main(String[] args) {
    Scanner scn = new Scanner(System.in);
    int n = scn.nextInt();
    int A[] = new int[n];
    for (int i = 0; i < n; i++) {
        A[i] = scn.nextInt();
    }
}
```



heap memory (60)

Print Array Left to Right

```
static void print1DArrayLR(int A[]) {
    for (int i = 0; i < A.length; i++) {
        System.out.print(A[i] + " ");
    }
    System.out.println();
}
```

Print Array Right to Left

```
static void print1DArrayRL(int A[]) {
    for (int i = A.length - 1; i >= 0; i--) {
        System.out.print(A[i] + " ");
    }
    System.out.println();
}
```


Sum of an array

A:

0	1	2	3	4	5
3	2	4	1	2	3
✓	✓	✓	✓	✓	✓

OP = 15

S:0 S:3 S:5 S:9 S:10 S:12 S:15

```
class Solution {  
    int arraySum(int arr[]) {  
        // code here  
        int sum=0;  
        for(int val:arr){  
            sum=sum+val;  
        }  
        return sum;  
    }  
}
```

Max Element in array

A:

0	1	2	3	4	5	6
3	1	7	11	15	19	10
✓	✓	✓	✓	✓	✓	✓

Max=3 Max=7 Max=11 Max=15 Max=19

if(val > max)
 max = val

```
int max=arr[0];  
for(int val:arr){  
    if(val>max) max=val;  
}  
return mas;
```

Second largest Element

A:

0	1	2	3	4	5	6	7
3	9	2	9	8	9	7	9
✓	✓	✓	✓	✓	✓	✓	✓

M1=9 M2=8

A:

0	1	2	3	4	5	6	7
9	9	9	9	9	9	9	9
✓	✓	✓	✓	✓	✓	✓	✓

M1=9 M2=-1

```
class Solution {  
    public int getSecondLargest(int[] arr) {  
        // code here  
        int max1=arr[0];  
        for(int val:arr){  
            max1=Math.max(max1,val);  
        }  
        int max2=-1;  
        for(int val:arr){  
            if(val!=max1){  
                max2=Math.max(max2,val);  
            }  
        }  
        return max2;  
    }  
}
```

Reverse an array

A:

0	1	2	3	4	5	6
8	2	3	1	9	7	5
✓	✓	✓	✓	✓	✓	✓

Exp

A:

0	1	2	3	4	5	6
8	2	3	1	9	7	5
✓	✓	✓	✓	✓	✓	✓

```
int i=0,j=arr.length-1;  
while(i<j){  
    int temp=arr[i];  
    arr[i]=arr[j];  
    arr[j]=temp;  
    i++;  
    j--;  
}
```


Array Palindrome

i *i* *i* *i < j* *j* *j* *j*

0 1 2 3 4 5 6

A =

7	9	8	2	8	9	7
---	---	---	---	---	---	---

```
public static boolean isPerfect(int[] arr) {  
    // code here  
    int i=0,j=arr.length-1;  
    while(i<j){  
        if(arr[i]!=arr[j]) return false;  
        i++;  
        j--;  
    }  
    return true;  
}
```